

VERIFIED SOURCE-GROUNDED STUDY HANDBOOK

Python + Backend + RAG + Agentic AI Project Code Handbook

Learning Python by Reading the Lenny Growth Assistant
Codebase

This handbook teaches Python through the current repository: its FastAPI request lifecycle, PostgreSQL/pgvector persistence, transcript ingestion, embeddings, retrieval, Ollama and Claude execution paths, deterministic agent routing, skills, tests, and Docker workflow.

Accuracy rule. Code examples are short excerpts verified against the current repository. When a helper is present but not on the active execution path, it is marked **documentation/reference only**. No secrets, tokens, passwords, transcript contents, or private environment values are included.

Current project facts: Python 3.14.4 on the host; Python 3.11 in the backend image; 21 backend tests currently pass; Ollama is the default provider; embedding model is all-MiniLM-L6-v2 with 384 dimensions.

Table of Contents

1. [How to use this handbook](#)
2. [A. Python environment](#)
3. [B. Project structure and execution map](#)
4. [C. Python constructs in this codebase](#)
5. [D. Actual package reference](#)
6. [E. Function-by-function reference](#)
7. [F. Class-by-class reference](#)
8. [G. FastAPI Python code](#)
9. [H. Database Python code](#)
10. [I. RAG Python code](#)
11. [J. Agentic AI Python code](#)
12. [K. LLM provider Python code](#)
13. [L. Testing Python](#)
14. [M. Error and debugging handbook](#)
15. [N. Complete execution map](#)
16. [O. Package to project-role map](#)
17. [P. Function sequence map](#)
18. [Q. Read the code like a beginner](#)
19. [R. Memorize versus understand](#)
20. [S. Interview preparation](#)
21. [Master checkpoint](#)

Source inventory covers backend/app, backend/tests, backend/alembic, backend requirements/configuration, Docker configuration, and the preserved legacy app/ directory. The transcript dataset, vectorstore, node_modules, and virtual-environment packages are not treated as project source.

How to Use This Handbook

Read in execution order. For each concept, first read the short source snippet, then follow the checkpoint questions. Finally open the referenced file and trace the next call. The goal is not to memorize the whole repository; it is to connect Python syntax to the RAG request lifecycle.

React → FastAPI route → Pydantic validation → SQLAlchemy session → GrowthAgent → skill → RAGService → pgvector retrieval → LLM executor → persisted messages → JSON response

Verified versus reference-only code

Area	Current status
Active ingestion discovery	<code>backend/app/ingestion/run_ingestion.py</code> uses <code>TRANSCRIPTS_ROOT.glob("*/transcript.md")</code> .
Loader discovery helper	<code>backend/app/ingestion/loader.py::find_transcript_files</code> still contains <code>episodes/*/transcript.md</code> . Documentation/reference only unless called directly; the active runner does not call it.
Legacy Streamlit app	<code>app/app.py</code> and <code>app/functions.py</code> are preserved reference code, not the active Docker web path.
Claude SDK	Installed and structurally wired through <code>ClaudeAgentExecutor</code> ; real cloud authentication is external and not part of the default Ollama workflow.

A. Python Environment

Host: the verified host interpreter reports Python 3.14.4. The repository contains `myenv/`, a local virtual environment ignored by Git. **Container:** `backend/Dockerfile` uses `python:3.11-slim`, installs `backend/requirements.txt`, and runs Uvicorn. The host environment and container environment are separate; Docker is authoritative for backend tests and runtime.

```
# Useful local inspection commands
python --version
py --version
python -m venv myenv
.\myenv\Scripts\Activate.ps1
python -m pip install -r backend\requirements.txt
python -m pip list
pip show fastapi

# Authoritative project commands
docker compose up -d --build backend
docker compose exec backend python -m pytest -q
```

CONCEPT: Python environment, virtualenv, pip, pinned requirements	
1. WHAT IS IT?	A Python interpreter plus an isolated package environment. <code>pip</code> installs packages listed in a requirements file.
2. WHAT DOES IT DO?	It makes imports such as FastAPI, SQLAlchemy, pgvector, and sentence-transformers available to the process.
3. WHY DID I USE IT IN THIS PROJECT?	RAG and API behavior depend on compatible versions across the backend image and tests.
4. HOW DID I USE IT?	Versions are pinned in <code>backend/requirements.txt</code> ; Docker installs them during image build.
5. WHERE IS IT USED?	<code>backend/Dockerfile</code> , <code>backend/requirements.txt</code> , ignored <code>myenv/</code> .
6. HOW DOES IT CONNECT TO THE REST OF THE PROJECT?	The interpreter loads the application modules and their third-party dependencies.
7. HOW DOES IT HELP COMPLETE THE RAG PROJECT?	It supplies the embedding, chunking, database, API, and testing libraries.
8. WHY DID WE USE THIS APPROACH?	Reproducible Docker builds and explicit dependency versions are easier to validate than an unmanaged global environment.
9. WHY NOT THE MAIN ALTERNATIVES?	A system-wide install risks conflicts; an unpinned requirements file risks drift.
10. WHAT WOULD BREAK IF IT WERE REMOVED?	Imports fail and the backend image cannot start or test.
11. COMMON FAILURE / ERROR	Dependency resolution, Python-version, CPU-wheel, or package API incompatibility.
12. HOW TO DEBUG OR TEST IT	Run <code>python --version</code> , <code>pip show</code> , rebuild Docker, and run pytest inside the backend container.
13. INTERVIEW ANSWER	“I separate host tooling from a pinned Python 3.11 Docker runtime so the API and RAG dependencies are reproducible.”

B. Project Structure and Execution Flow

Path	Role and next call
backend/app/main.py	Creates FastAPI, loads environment variables, configures CORS, includes API routes, and exposes health/root endpoints.
backend/app/api/routes.py	Validates requests through schemas, gets DB sessions, persists messages, calls <code>GrowthAgent</code> , and returns JSON.
backend/app/api/schemas.py	Pydantic request/response models and field validation.
backend/app/db/database.py	Creates SQLAlchemy engine/session factory and FastAPI database dependency.
backend/app/db/models.py	ORM tables, relationships, constraints, and pgvector column.
backend/app/ingestion/	Loads transcripts, cleans, chunks, embeds, stores, and retrieves vectors.
backend/app/services/rag_service.py	Maps retrieved rows to sources, builds context, and asks the LLM for grounded answers.
backend/app/llm/	Defines provider interface, Ollama, Hugging Face, and Claude executor compatibility.
backend/app/agent/	Routes user intent and invokes RAG, Ship30, or Artifact skills.
backend/tests/	Health, persistence, API, routing, artifact security, Claude-structure, and retrieval-contract tests.

```
main.py
-> api.routes.chat()
  -> GrowthAgent.execute()
    -> AgentRouter.route()
    -> RAGSkill / Ship30Skill / ArtifactSkill
      -> RAGService.retrieve()
        -> TranscriptRetriever.search()
          -> TranscriptEmbedder.embed_text()
            -> PostgreSQL pgvector SQL
          -> LLMProvider.generate()
        -> Message commit + ChatResponse
```

C. Python Constructs Actually Used

Variables, strings, lists, dictionaries, tuples, booleans, and None

```
messages = []
payload = {"model": self.model, "messages": messages, "stream": False}
artifact = result.get("artifact")
```

CONCEPT: Python collections and None

1. WHAT IS IT?	Variables bind names to objects; lists are ordered mutable collections; dictionaries map keys to values; tuples are fixed ordered values; <code>None</code> represents absence.
2. WHAT DOES IT DO?	The provider builds a list of chat messages and a JSON-compatible dictionary.
3. WHY DID I USE IT?	HTTP payloads and source records are naturally represented as nested Python collections.
4. HOW DID I USE IT?	<code>messages.append(...)</code> , dictionary literals, <code>result.get(...)</code> , and optional artifact values.
5. WHERE IS IT USED?	<code>ollama_provider.py</code> , <code>rag_service.py</code> , <code>routes.py</code> , <code>schemas/tests</code> .
6. HOW DOES IT CONNECT?	Collections carry user input, prompts, source metadata, and API serialization between layers.
7. HOW DOES IT HELP RAG?	Retrieved chunks become source dictionaries and prompt context.
8. WHY THIS APPROACH?	It matches JSON and SQL result shapes without unnecessary wrapper classes for every intermediate value.
9. WHY NOT ALTERNATIVES?	A custom object for every payload would add ceremony; raw strings would lose structure.
10. WHAT BREAKS?	Malformed keys or missing values can produce invalid prompts, responses, or provider payloads.
11. COMMON FAILURE / ERROR	<code>KeyError</code> , wrong value types, mutable-list reuse, or <code>None</code> passed to code expecting text.
12. DEBUG / TEST	Print safe shape information, assert keys in tests, and inspect response models.
13. INTERVIEW ANSWER	“The project uses dictionaries and lists at integration boundaries because providers and HTTP responses are JSON-shaped.”

Functions, parameters, defaults, returns, kwargs, and lambdas

```
def search(self, db: Session, query: str, top_k: int = 5,
          min_similarity: float = 0.45) -> list[dict]:
    ...

    lambda **kwargs: []
```

CONCEPT: functions and typed parameters	
1. WHAT IS IT?	A reusable block with named inputs and a return contract.
2. WHAT DOES IT DO?	<code>search</code> accepts a DB session and query options, then returns row dictionaries.
3. WHY DID I USE IT?	Functions isolate ingestion, retrieval, API, provider, and skill responsibilities.
4. HOW DID I USE IT?	Required parameters, default <code>top_k</code> / <code>min_similarity</code> , annotations, and explicit returns.
5. WHERE IS IT USED?	Every active backend layer; tests also use lambdas as small fake collaborators.
6. HOW DOES IT CONNECT?	Each function is a call-graph edge, such as <code>route</code> → <code>agent</code> → <code>skill</code> → <code>service</code> .
7. HOW DOES IT HELP RAG?	It makes each RAG stage independently testable.
8. WHY THIS APPROACH?	Small functions make side effects and inputs visible.
9. WHY NOT ALTERNATIVES?	A single monolithic request function would be harder to test and reason about.
10. WHAT BREAKS?	Changing a signature without updating callers causes runtime or type errors.
11. COMMON FAILURE / ERROR	Wrong argument names, missing dependencies, incorrect return shape.
12. DEBUG / TEST	Use focused unit tests with fake DB/LLM objects and inspect call arguments.
13. INTERVIEW ANSWER	“Typed functions make the pipeline explicit: query in, embeddings and rows through retrieval, structured answer out.”

Classes, self, constructors, inheritance, ABC, dataclasses, and static methods

```

class LLMPProvider(ABC):
    @abstractmethod
    def generate(self, prompt: str, system_prompt: str | None = None) -> str:
        raise NotImplementedError

    @dataclass(frozen=True)
    class AgentRoute:
        skill: str
        reason: str

class OllamaProvider(LLMPProvider):
    def __init__(self) -> None:
        self.base_url = os.getenv("OLLAMA_BASE_URL", ...)

```

CONCEPT: objects, inheritance, abstract interfaces, dataclasses	
1. WHAT IS IT?	A class defines object behavior; inheritance reuses/interface-matches behavior; an ABC declares required methods; a dataclass stores structured values.
2. WHAT DOES IT DO?	<code>LLMProvider</code> standardizes <code>generate</code> ; Ollama and Claude implement it; <code>AgentRoute</code> stores route data.
3. WHY DID I USE IT?	Skills can depend on one LLM interface while the concrete provider changes.
4. HOW DID I USE IT?	Constructors assign configuration; methods use <code>self</code> ; providers inherit from <code>LLMProvider</code> .
5. WHERE IS IT USED?	<code>backend/app/llm</code> , <code>agent/router.py</code> , services and skills.
6. HOW DOES IT CONNECT?	The agent injects an LLM object into RAG/Ship30/Artifact skills.
7. HOW DOES IT HELP RAG?	RAG generation is decoupled from the specific model transport.
8. WHY THIS APPROACH?	Polymorphism allows Ollama by default and Claude/Hugging Face as alternatives.
9. WHY NOT ALTERNATIVES?	Hard-coding HTTP calls in every skill would duplicate provider logic.
10. WHAT BREAKS?	A provider that fails to implement <code>generate</code> cannot satisfy the interface.
11. COMMON FAILURE / ERROR	Wrong provider selection, missing environment variables, or incompatible response shapes.
12. DEBUG / TEST	Inject <code>FakeLLM</code> ; test one interface call and provider-specific configuration.
13. INTERVIEW ANSWER	“The abstract provider is a narrow polymorphic contract; skills depend on generation, not on Ollama’s transport details.”

Exceptions, try/except, context managers, logging, and decorators

```
try:
    db.add(user_message)
    result = growth_agent.execute(db=db, message=request.message)
    db.commit()
except SQLAlchemyError:
    db.rollback()
    logger.exception("Database failure while processing chat request")
    raise HTTPException(status_code=503, ...)

with engine.connect() as connection:
    connection.execute(text("SELECT 1"))
```

CONCEPT: error handling and resource management	
1. WHAT IS IT?	Exceptions represent failures; <code>try/except</code> handles them; <code>with</code> manages setup/cleanup.
2. WHAT DOES IT DO?	The chat route rolls back partial DB work and returns a safe 503; health closes its DB connection.
3. WHY DID I USE IT?	LLM/network/database failures are expected operational conditions.
4. HOW DID I USE IT?	Catch SQLAlchemy and general agent failures, log stack traces server-side, rollback, raise HTTPException.
5. WHERE IS IT USED?	<code>routes.py</code> , <code>main.py</code> , <code>database.py</code> , provider code.
6. HOW DOES IT CONNECT?	It protects transaction boundaries around agent execution.
7. HOW DOES IT HELP RAG?	A failed Ollama call cannot leave a user-only message committed.
8. WHY THIS APPROACH?	Users get a stable API error while operators retain diagnostics.
9. WHY NOT ALTERNATIVES?	Ignoring exceptions risks corrupted state; exposing raw traces leaks internals.
10. WHAT BREAKS?	Without rollback, later queries may see a failed transaction or partial turn.
11. COMMON FAILURE / ERROR	Ollama HTTP 500, PostgreSQL connection failure, malformed provider response.
12. DEBUG / TEST	<code>test_chat_failure_returns_service_unavailable_and_rolls_back_user_message</code> monkeypatches the agent.
13. INTERVIEW ANSWER	“The API treats generation as part of a DB transaction and converts operational failures into a safe 503.”

Async/await, coroutines, generators, comprehensions, imports, paths, and JSON

```
@asynccontextmanager
async def lifespan(_: FastAPI):
    yield

async def _generate(...):
    async for message in query(prompt=prompt, options=options):
        ...

sources = [
    {"chunk_id": str(result["id"]), ...}
    for result in results
]
```

Async appears in FastAPI lifespan and the Claude SDK executor. The main API endpoints and Ollama HTTP call are synchronous. `yield` appears in the DB dependency and async iteration over Claude messages. Comprehensions build source lists. `Path` , `os.getenv` , `json.dumps` / `loads` , and imports connect filesystem/configuration/network boundaries.

D. Actual Package Reference

Versions below come from `backend/requirements.txt`. “Actual import” means the package was found in current backend source; a package listed but not directly imported is marked reference/dependency-only.

Package / version	Actual import and files	Project role
FastAPI 0.115.2	<code>FastAPI</code> , <code>APIRouter</code> , dependencies, middleware in <code>main.py</code> / <code>routes.py</code>	HTTP application and endpoints
Uvicorn 0.32.0	Runtime command in Dockerfile; no Python import	ASGI server; runtime/configuration
Pydantic 2.12.5	<code>BaseModel</code> , <code>Field</code> , <code>Literal</code> , validators in <code>schemas.py</code>	Request validation and response serialization
SQLAlchemy 2.0.36	Engine/session ORM, <code>select</code> , <code>text</code> , mapped columns	Database connection, ORM, transactions, SQL
psycopg 3.2.13	URL driver in <code>postgresql+psycopg</code> ; no direct Python import	PostgreSQL driver
Alembic 1.14.0	<code>op</code> , <code>context</code> , <code>command</code> in migrations/tests	Schema versioning
pgvector 0.3.6	<code>Vector</code> ORM type and SQL operator query	384-D vector storage/search
sentence-transformers 6.0.1	<code>SentenceTransformer</code> in <code>embedder.py</code>	Text to normalized embedding
torch 2.8.0+cpu	Transitive/runtime backend for sentence-transformers; no direct project import	CPU model execution
langchain-text-splitters 1.1.2	<code>RecursiveCharacterTextSplitter</code> in <code>chunker.py</code>	Overlap-aware transcript chunks
PyYAML 6.0.3	<code>yaml.safe_load</code> in <code>loader.py</code>	Transcript front matter
python-dotenv 1.0.1	<code>load_dotenv</code> in <code>main.py</code> , <code>database.py</code> , <code>alembic/env.py</code>	Local environment loading
claude-agent-sdk 0.2.152	Lazy imports in <code>agent/executor.py</code>	Optional Claude application-agent execution
pytest 8.3.3	Tests and fixtures in <code>backend/tests</code>	Automated verification
httpx 0.27.2	<code>TestClient</code> dependency; no direct test import	HTTP testing support

Reference-only distinction: Hugging Face’s `InferenceClient` is directly used in `huggingface_provider.py` , but the default runtime is Ollama. Other large packages in the legacy `app/requirements.txt` are not part of the active backend Docker path unless imported by the preserved Streamlit application.

Package checkpoint example: FastAPI

PACKAGE: FastAPI 0.115.2 · CATEGORY: web framework	
1. WHAT IS IT?	A Python ASGI web framework.
2. WHAT DOES IT DO?	Maps HTTP requests to typed Python functions and serializes responses.
3. WHY DID I USE IT?	The project needs health, session, message, and chat APIs.
4. HOW DID I USE IT?	<code>FastAPI()</code> , <code>APIRouter</code> , <code>Depends</code> , <code>HTTPException</code> , response models.
5. WHERE IS IT USED?	<code>main.py</code> , <code>routes.py</code> , tests.
6. HOW DOES IT CONNECT?	It is the boundary between React and Python orchestration.
7. HOW DOES IT HELP RAG?	It exposes the chat request that starts retrieval and generation.
8. WHY THIS APPROACH?	Type hints and Pydantic integration reduce request-validation boilerplate.
9. WHY NOT ALTERNATIVES?	A lower-level WSGI/ASGI setup would require more manual validation and routing.
10. WHAT WOULD BREAK?	No API surface for the frontend.
11. COMMON FAILURE / ERROR	Dependency mismatch, bad response model, CORS or request validation errors.
12. HOW TO DEBUG OR TEST IT	Call <code>/health</code> , use <code>TestClient</code> , inspect status codes and JSON.
13. INTERVIEW ANSWER	“FastAPI gives the project typed, testable HTTP endpoints and dependency injection around the DB session.”

E. Function-by-Function Reference

Function	Inputs → output	Called by / next	RAG role
health main.py	HTTP request → HealthResponse	HTTP client → DB probe	Operational readiness
get_db database.py	none → generator of Session	FastAPI Depends → route	DB access for retrieval/persistence
chat routes.py	ChatRequest, DB → ChatResponse	frontend → GrowthAgent → commit	Complete request boundary
load_transcript loader.py	Path → Transcript dataclass	ingest_episode	Document metadata/text input
clean_transcript cleaner.py	str → str	ingest_episode → chunker	Removes timestamps/blank lines
chunk_transcript chunker.py	text, size, overlap → list[str]	ingest_episode → embedder	Creates embedding units
embed_text/embed_texts embedder.py	text(s) → float list(s)	ingestion/retriever	Vector representation
save_transcript repository.py	DB, Transcript, chunks, embeddings → Document	ingest_episode → commit	Stores searchable knowledge
search retriever.py	DB, query, top_k, threshold → row dicts	RAGService.retrieve	Vector similarity retrieval
retrieve/answer rag_service.py	DB, question → sources/context/answer	skills → provider	Grounding and context construction
get_llm_provider factory.py	environment → LLMProvider	agent/RAG service	Model selection
generate ollama_provider.py	prompt/system → str	RAG/Ship30/Artifact	Answer generation
route router.py	message → AgentRoute	GrowthAgent	Skill selection
execute agent.py	DB, message → result dict	chat route	Orchestration
normalize_html_artifact/_sanitize artifact_skill.py	HTML str → safe-ish HTML str	ArtifactSkill.execute	Artifact response safety

Function checkpoint: `TranscriptRetriever.search`

Actual location: <code>backend/app/ingestion/retriever.py</code>	
1. WHAT IS IT?	A method that embeds a query and runs a vector similarity SQL query.
2. WHAT DOES IT DO?	Orders chunks by pgvector distance and filters by <code>min_similarity</code> .
3. WHY DID WE CREATE IT?	To isolate database retrieval from prompt/generation logic.
4. HOW DOES IT WORK?	Embed query; bind vector, threshold, and limit; execute SQL; map rows to dictionaries.
5. WHERE IS IT CALLED?	<code>RAGService.retrieve</code> .
6. WHAT DOES IT CALL NEXT?	SQLAlchemy <code>db.execute</code> , then RAG source mapping.
7. HOW DOES IT CONNECT TO RAG?	It supplies the evidence that prevents unsupported answers.
8. WHY THIS DESIGN?	It keeps vector SQL in one owning abstraction.
9. ALTERNATIVE?	Use an ORM vector expression or external vector database; current Postgres is already the persistence layer.
10. WHAT BREAKS?	Questions have no transcript context.
11. COMMON FAILURE	Dimension mismatch, invalid vector cast, connection failure, or threshold too high.
12. TEST/DEBUG	<code>tests/test_retrieval.py</code> uses controlled rows and asserts threshold/metadata.
13. INTERVIEW ANSWER	“The retriever embeds the query and lets pgvector rank and threshold stored chunks before RAG builds the prompt.”

F. Class-by-Class Reference

Class	File / base	Responsibility / important methods
HealthResponse	main.py / Pydantic BaseModel	Health serialization: status, service, database.
SessionCreate, MessageCreate, ChatRequest	schemas.py / BaseModel	Validate request bodies with Field and field_validator.
ChatResponse, ChatSource, ArtifactResponse	schemas.py / BaseModel	Define API output shape.
User, Session, Message, Document, Chunk	db/models.py / SQLAlchemy Base	ORM tables, relationships, constraints, vector column.
Transcript	loader.py / dataclass	Typed metadata/text document object.
TranscriptEmbedder	embedder.py	Owens SentenceTransformer and encoding methods.
TranscriptRetriever	retriever.py	Owens query embedding and pgvector search.
RAGService	rag_service.py	Retrieval-to-context-to-answer orchestration.
LLMProvider	provider.py / ABC	Abstract generate contract.
OllamaProvider	ollama_provider.py / LLMProvider	HTTP JSON chat to Ollama.
ClaudeAgentExecutor	agent/executor.py / LLMProvider	Optional SDK query path.
GrowthAgent	agent.py	Chooses route, creates executor/skill, returns result.
AgentRouter / AgentRoute	router.py / dataclass route	Deterministic intent classification.
RAGSkill, Ship30Skill, ArtifactSkill	agent/skills	Explicit skill boundaries; all reuse retrieval as appropriate.
RecordingSkill, FakeRAGService, FakeLLM, FakeDatabase	backend/tests	Test doubles; not production classes.

CLASS CHECKPOINT: <code>GrowthAgent</code>	
1. WHAT IS IT?	The application orchestration object for a chat request.
2. WHAT DOES IT DO?	Routes input, creates the selected execution path and skill, invokes it, adds metadata.
3. WHY DID WE USE IT?	To keep API routes thin and make skills explicit.
4. HOW DID WE USE IT?	Constructor accepts an optional router and skill dictionary for dependency injection; <code>execute</code> handles dispatch.
5. WHERE IS IT USED?	Global <code>growth_agent</code> in <code>api/routes.py</code> .
6. HOW DOES IT CONNECT?	It is between HTTP and skills/providers.
7. HOW DOES IT HELP RAG?	It selects RAGSkill without placing routing logic inside RAGService.
8. WHY THIS APPROACH?	Deterministic orchestration is explainable and testable.
9. WHY NOT A FULL AUTONOMOUS AGENT?	The assignment needs predictable skill routing; a full planner would add latency and nondeterminism.
10. WHAT BREAKS?	Chat cannot choose or execute a skill.
11. COMMON FAILURE	Incorrect provider selection or skill result shape.
12. DEBUG/TEST	Inject <code>RecordingSkill</code> ; inspect route and calls in <code>test_agent.py</code> .
13. INTERVIEW ANSWER	“GrowthAgent is a small deterministic application agent: it routes to explicit skills and injects either local or Claude execution.”

G. FastAPI Python Code

Method/path	Function and input	Output/action
GET /	<code>root()</code>	Service status dictionary.
GET /health	<code>health()</code>	DB SELECT 1; HealthResponse.
POST /api/sessions	<code>create_session(SessionCreate, db=Depends(get_db))</code>	Create/find User, create Session, 201.
GET /api/sessions/{session_id}	<code>get_session</code>	Session or 404.
GET /api/sessions/{id}/messages	<code>get_messages</code>	Ordered Message list.
POST /api/sessions/{id}/messages	<code>create_message(MessageCreate)</code>	Persist one message, 201.
POST /api/chat	<code>chat(ChatRequest)</code>	Transactionally persist user/assistant turn and return answer, sources, skill, route reason, artifact.

```

POST /api/chat
-> Pydantic ChatRequest
-> db.get(Session, session_id)
-> db.add(user_message); db.flush()
-> growth_agent.execute(db, message)
    -> router -> skill -> retrieval -> LLM
-> db.add(assistant_message); db.commit()
-> ChatResponse model serialization

```

CONCEPT: FastAPI dependency injection and Pydantic validation	
1. WHAT IS IT?	FastAPI injects declared dependencies; Pydantic validates typed data models.
2. WHAT DOES IT DO?	<code>Depends(get_db)</code> supplies a DB session; request fields enforce length, role, and nonblank values.
3. WHY DID I USE IT?	It keeps endpoint code focused on workflow rather than parsing JSON manually.
4. HOW DID I USE IT?	Annotations, <code>Field</code> , <code>field_validator</code> , response models, status codes.
5. WHERE?	<code>main.py</code> , <code>routes.py</code> , <code>schemas.py</code> .
6. CONNECTION?	Validated HTTP data becomes typed Python arguments for DB and agent code.
7. RAG HELP?	It protects the RAG pipeline from blank/oversized questions and invalid session IDs.
8. WHY?	Explicit contracts make frontend/backend integration predictable.
9. WHY NOT MANUAL JSON?	Manual parsing duplicates validation and produces inconsistent errors.
10. BREAKAGE?	Bad input reaches SQL/provider code or response serialization fails.
11. COMMON ERROR?	422 validation errors, missing dependency, response model mismatch.
12. DEBUG?	Use <code>TestClient</code> and assert status/JSON in persistence tests.
13. INTERVIEW?	“FastAPI turns Python type annotations into validated, documented API boundaries and injects the DB session per request.”

H. Database Python Code

```
engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db() -> Generator:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

SQLAlchemy creates the engine, session factory, declarative base, ORM classes, relationships, and transaction methods. Psycopg is the driver selected by `postgresql+psycopg`. `pgvector`'s `Vector(dim=384)` maps embedding lists to a PostgreSQL vector column. The retriever uses SQLAlchemy `text()` because the `pgvector` distance operator is expressed directly in SQL.

Python list[float] → SQLAlchemy bind parameter → CAST(... AS vector) → PostgreSQL pgvector cosine distance → similarity float → source dict → prompt

CONCEPT: ORM model, transaction, relationship, pgvector

1. WHAT IS IT?	An ORM maps Python classes/attributes to database tables/columns; a transaction groups changes; <code>pgvector</code> stores/searches vectors.
2. WHAT DOES IT DO?	Stores users, sessions, messages, documents, chunks and embeddings.
3. WHY?	RAG needs durable document evidence and chat needs durable conversation state.
4. HOW?	<code>Mapped</code> , <code>mapped_column</code> , <code>relationships</code> , <code>db.add</code> , <code>flush</code> , <code>commit</code> , <code>rollback</code> .
5. WHERE?	<code>database.py</code> , <code>models.py</code> , <code>repository.py</code> , <code>routes</code> , <code>retriever</code> .
6. CONNECTION?	DB rows are the bridge between ingestion and RAG answer generation.
7. RAG?	Embeddings and source metadata survive process restarts.
8. WHY?	PostgreSQL already provides relational persistence and <code>pgvector</code> extension support.
9. ALTERNATIVES?	External vector stores or raw SQL everywhere would split persistence or increase boilerplate.
10. BREAKAGE?	No persisted source corpus, sessions, or similarity search.
11. COMMON FAILURE?	Connection errors, migration mismatch, vector dimension mismatch, uncommitted transaction.
12. DEBUG?	Health endpoint, Alembic current/upgrade, SQLAlchemy test DB, database counts.
13. INTERVIEW?	“PostgreSQL is both the system of record and vector index; SQLAlchemy owns transactions while <code>pgvector</code> ranks chunks.”

I. RAG Python Code

Stage	Input → function/package → output
Discovery	Mounted root → <code>run_ingestion.main</code> <code>glob */transcript.md</code> → Paths.
Load metadata	Path → <code>load_transcript</code> + PyYAML → Transcript dataclass.
Clean	Transcript text → <code>clean_transcript</code> + regex → normalized text.
Chunk	Text → <code>RecursiveCharacterTextSplitter</code> → overlapping chunks.
Embed	Chunks → <code>SentenceTransformer</code> → normalized 384-D lists.
Store	Transcript/chunks/vectors → <code>save_transcript</code> → Document/Chunk rows.
Query	Question → <code>embed_text</code> → query vector.
Retrieve	Vector → pgvector SQL → rows above 0.45.
Ground	Rows → source dictionaries/context string → prompt.
Generate	Prompt → LLM provider → answer.
Insufficient evidence	No sources → fixed no-support message; no LLM call.

```

1 - (c.embedding <=> CAST(:query_embedding AS vector)) AS similarity
WHERE c.embedding IS NOT NULL
AND 1 - (c.embedding <=> CAST(:query_embedding AS vector)) >= :min_similarity

```

CONCEPT: grounding and similarity threshold	
1. WHAT IS IT?	RAG retrieves evidence before generation; similarity estimates vector closeness.
2. WHAT DOES IT DO?	Only chunks at or above 0.45 become sources/context.
3. WHY?	To reduce unsupported claims and provide inspectable evidence.
4. HOW?	Retriever returns rows; RAGService maps metadata and joins context.
5. WHERE?	<code>retriever.py</code> and <code>rag_service.py</code> .
6. CONNECTION?	It connects database vectors to provider prompts.
7. RAG?	This is the central retrieve-then-generate pattern.
8. WHY?	Semantic search handles paraphrases better than exact keyword matching.
9. ALTERNATIVE?	Keyword search, external vector DB, or hybrid search could be added later.
10. BREAKAGE?	Irrelevant context, hallucination risk, or empty answers.
11. COMMON FAILURE?	Wrong model dimension, threshold too high/low, poor chunking, empty corpus.
12. DEBUG?	Controlled threshold tests, inspect source similarities and database counts, test unrelated queries.
13. INTERVIEW?	“I embed both corpus chunks and the query into the same 384-D space, then use pgvector distance plus a threshold before prompting the LLM.”

J. Agentic AI Python Code

```
route = self.router.route(message)
if route.skill == "rag":
    result = skill.execute(db=db, question=message)
elif route.skill == "ship30":
    result = skill.execute(db=db, topic=message)
else:
    result = skill.execute(db=db, request=message)
```

The project's agent is intentionally deterministic. The router checks artifact terms first, Ship30 terms second, and defaults to RAG. The skill performs LLM work; routing itself does not ask a model to decide. In Claude mode the application agent creates `ClaudeAgentExecutor`, which invokes the SDK. In Ollama mode it creates the configured local provider.

Skill	Input and behavior	Evidence
RAGSkill	Question → <code>RAGService.answer</code>	<code>agent/skills/rag_skill.py</code>
Ship30Skill	Topic → retrieval context → approximately 1,250-word grounded essay	<code>ship30_skill.py</code>
ArtifactSkill	Request → retrieval context → HTML normalization/sanitization → artifact object	<code>artifact_skill.py</code>

CONCEPT: deterministic agent routing and skill boundaries

1. WHAT IS IT?	An application-level orchestrator that selects a specialized operation.
2. WHAT DOES IT DO?	Chooses and invokes one explicit skill, then adds route metadata.
3. WHY?	Separate answer types have different prompts and output contracts.
4. HOW?	Dataclass route + skill dictionary injection + lazy construction.
5. WHERE?	<code>agent.py</code> , <code>router.py</code> , <code>agent/skills</code> .
6. CONNECTION?	API calls agent; agent calls skills; skills call RAG/provider.
7. RAG?	RAG is one skill and a shared evidence service for generation skills.
8. WHY?	Predictability, transparent reasons, cheap tests.
9. ALTERNATIVE?	LLM planner or fully autonomous tool loop; not chosen for this small take-home.
10. BREAKAGE?	All requests would need one monolithic prompt handler.
11. COMMON FAILURE?	Ambiguous substring routing or skill result mismatch.
12. DEBUG?	Route unit tests, injected <code>RecordingSkill</code> , live API skill field.
13. INTERVIEW?	“Agentic behavior here means explicit orchestration of specialized skills; the local path is deterministic, while the selected executor supplies generation.”

K. LLM Provider Python Code

```
class LLMPProvider(ABC):
    @abstractmethod
    def generate(self, prompt: str,
                 system_prompt: str | None = None) -> str:
        raise NotImplementedError

provider = os.getenv("LLM_PROVIDER", "ollama").lower()
if provider == "ollama":
    return OllamaProvider()
if provider in {"claude", "claude-agent-sdk"}:
    return ClaudeAgentProvider()
```

`OllamaProvider.generate` builds JSON and calls `/api/chat` with `urllib.request.urlopen`. `ClaudeAgentExecutor` lazily imports the SDK, creates `ClaudeAgentOptions`, asynchronously iterates `query()`, and joins assistant text blocks. `HuggingFaceProvider` uses `InferenceClient` when explicitly selected and requires `HF_TOKEN`.

CONCEPT: abstraction, polymorphism, environment-selected dependency injection

1. WHAT IS IT?	A common interface with interchangeable implementations.
2. WHAT DOES IT DO?	Lets skills call <code>generate</code> without knowing transport/model details.
3. WHY?	Ollama is mandatory local/demo behavior; cloud paths are optional.
4. HOW?	Factory selects provider; GrowthAgent selects Claude executor explicitly in Claude mode.
5. WHERE?	<code>llm/provider.py</code> , factory, provider classes, agent executor.
6. CONNECTION?	RAG/Ship30/Artifact prompts end at the interface.
7. RAG?	Generation is the final stage after retrieved evidence.
8. WHY?	Switching models should not rewrite skills.
9. ALTERNATIVE?	Direct provider calls in every skill would duplicate integration code.
10. BREAKAGE?	Provider errors or wrong output shape stop answer generation.
11. COMMON FAILURE?	Ollama unavailable, HTTP 500, missing HF token, Claude authentication failure.
12. DEBUG?	Print only safe provider name/base URL/model; mock interface in tests; inspect 503 logs.
13. INTERVIEW?	“The provider contract is a stable seam between domain skills and model transports, with Ollama as the safe default.”

L. Testing Python

Test file	Category	What it protects
test_health.py	API smoke	App imports and health response shape.
test_persistence.py	PostgreSQL integration/API	Migrations, sessions, messages, isolation, validation, rollback, chat shape.
test_agent.py	Unit/mock	Routing, skill reuse, sanitizer, HTML normalization, Claude SDK structure.
test_retrieval.py	Controlled retrieval contract	Threshold, SQL operator, source metadata, no-support path.

```
@pytest.fixture(scope="module", autouse=True)
def ensure_test_database():
    command.upgrade(alembic_cfg, "head")
    yield

monkeypatch.setattr("app.api.routes.growth_agent.execute", fail_answer)
assert response.status_code == 503
```

CONCEPT: pytest fixtures, monkeypatch, TestClient, fakes

1. WHAT IS IT?	Pytest discovers functions; fixtures prepare resources; monkeypatch replaces behavior; TestClient sends HTTP calls in-process.
2. WHAT DOES IT DO?	Tests isolate expensive providers and exercise persistence against a test DB.
3. WHY?	Fast repeatable feedback without requiring Ollama for every test.
4. HOW?	FakeEmbedder, FakeDatabase, FakeLLM, RecordingSkill, fixture migration, assertions.
5. WHERE?	backend/tests .
6. CONNECTION?	Tests verify individual edges and the API transaction path.
7. RAG?	Retrieval threshold and source mapping are tested without downloading the model.
8. WHY?	Mocks make failure and route tests deterministic.
9. ALTERNATIVE?	Only live tests would be slow/flaky; only mocks would miss database behavior.
10. BREAKAGE?	Regressions in validation, persistence, routing, or security go unnoticed.
11. COMMON FAILURE?	Wrong test DB URL, stale Docker image, no PostgreSQL, model loading during collection.
12. DEBUG?	Run focused module first, rebuild image, then full <code>pytest -q</code> .
13. INTERVIEW?	"I combine real persistence tests with cheap fakes around embeddings/providers so the important contracts are covered without making the suite depend on Ollama."

M. Error and Debugging Handbook

Symptom	Likely cause	Where/command	Why the check helps
Backend cannot import	Wrong Python/dependencies or stale image	<code>docker compose up -d --build backend ; python --version</code>	Rebuilds the authoritative environment.
PostgreSQL connection failure	Service not healthy or wrong DATABASE_URL	<code>docker compose ps ; /health ; logs</code>	Checks dependency and DB connectivity.
Vector dimension error	Model 384 but schema 1536 or vice versa	<code>alembic current ; models/migrations</code>	All stored/query vectors must share 384 dimensions.
No transcript files	Wrong mount root/glob	Inspect <code>/app/lenny_transcripts ;</code> runner uses <code>*/transcript.md</code>	Host episodes directory is mounted directly.
No RAG sources	Empty corpus or threshold too high	Database counts; <code>test_retrieval.py ; similarities</code>	Separates storage from ranking behavior.
Ollama HTTP error/timeout	Ollama/model unavailable or local load	<code>ollama list ;</code> provider env; backend logs	Container reaches host through host.docker.internal.
Claude auth error	External Claude authentication absent	LLM_PROVIDER , Claude environment	Claude is optional; Ollama remains fallback/default.
Raw fenced artifact	Model wrapped HTML in Markdown	ArtifactSkill normalization tests	Fences are stripped before sanitization/viewer.
Frontend cannot call API	VITE_API_BASE_URL/CORS/ports	Browser console, CORS settings, Compose	Confirms the browser origin is allowed.
Test file not found in container	Image not rebuilt after source/test edit	<code>docker compose up -d --build backend</code>	Container copies tests at image build time.

N. Complete Python Execution Map



Arrow	Responsible Python code
React → HTTP	<code>frontend/src/App.tsx</code> fetch calls.
HTTP → typed input	FastAPI route signature + Pydantic schemas.
Route → agent	<code>routes.chat</code> calls <code>growth_agent.execute</code> .
Agent → skill	<code>GrowthAgent.execute</code> dispatch.
Skill → evidence	RAGSkill/Ship3oSkill/ArtifactSkill call RAGService.
Evidence → DB	Retriever embeds and executes pgvector SQL.
Evidence → model	RAGService builds context and calls provider.
Result → persistence	Route adds assistant Message and commits transaction.
Result → UI	ChatResponse schema serializes sources/metadata/artifact.

O. Package → Function → Project Role Map

Package/import	Actual object/file	Role	RAG connection
fastapi	FastAPI, APIRouter, Depends, HTTPException	API and dependency injection	Starts request flow.
pydantic	BaseModel, Field, field_validator	Typed request/response contracts	Validates questions and sources.
sqlalchemy	create_engine, Session, select, text, mapped_column	ORM/SQL/transactions	Stores and retrieves chunks.
pgvector	Vector	Embedding column type	Enables similarity search.
sentence_transformers	SentenceTransformer	Embedding model	Maps text to vectors.
langchain_text_splitters	RecursiveCharacterTextSplitter	Chunking	Defines retrieval units.
yaml	safe_load	Transcript metadata	Preserves source metadata.
claude_agent_sdk	query, AssistantMessage, ClaudeAgentOptions	Optional Claude execution	Alternative final generation path.
huggingface_hub	InferenceClient	Optional provider	Alternative generation only.
dotenv	load_dotenv	Local config loading	Chooses DB/provider/model.
pytest	fixture, monkeypatch, assertions	Verification	Protects RAG contracts.
urllib.request/json	Request, urlopen, dumps/loads	Ollama HTTP transport	Generates grounded answer.

P. Function Sequence Map

Step	Function/class	Receives	Returns	Next
1	<code>main.app</code>	Environment	FastAPI app	Routes
2	<code>chat</code>	ChatRequest + DB	ChatResponse	GrowthAgent
3	<code>route</code>	message	AgentRoute	Skill
4	<code>execute</code>	DB + message	skill result	RAG/LLM
5	<code>retrieve</code>	question + DB	sources/context	Provider
6	<code>search</code>	query + DB	rows	source mapping
7	<code>embed_text</code>	query string	float vector	pgvector SQL
8	<code>generate</code>	prompt/system	answer string	route persistence
9	<code>commit</code>	messages	database state	JSON response

Q. Read the Code Like a Beginner

1. **requirements.txt**: identify runtime versus test/RAG packages; study imports and version pins.
2. **main.py**: see app creation, middleware, lifespan, health; study decorators and context managers.
3. **routes.py**: trace HTTP inputs and transaction boundaries; study dependency injection and exceptions.
4. **schemas.py**: study Pydantic fields, literals, validators, optional types.
5. **database.py/models.py**: study engine/session/ORM relationships and constraints.
6. **migrations**: compare Python schema history with current models and vector dimension.
7. **ingestion**: follow Path → Transcript → clean → chunks → embeddings → commit.
8. **retriever.py/rag_service.py**: follow vector SQL → sources → context → prompt.
9. **llm**: study ABC inheritance, factory selection, JSON HTTP, async SDK iteration.
10. **agent/skills**: study dataclass routing, dependency injection, explicit skill contracts.
11. **tests**: see real DB tests and controlled fakes; understand what is and is not integration-tested.

Before moving on, be able to answer: What enters this file? What leaves it? What side effect does it own? What evidence or state does it pass to the next layer?

R. Memorize versus Understand

MUST MEMORIZE

- Python functions, classes, lists/dicts, exceptions.
- Type hints and optional values.
- FastAPI route and Pydantic patterns.
- SQLAlchemy session/commit/rollback basics.
- `async` / `await` basics.
- Environment variable and Docker commands.
- Pytest assertions and fixtures.

MUST UNDERSTAND

- Chunk → embedding → vector retrieval.
- Similarity threshold and evidence grounding.
- Provider abstraction and model selection.
- Agent routing and explicit skills.
- Transaction boundary around generation.
- Container networking to host Ollama.
- Why tests use fakes for expensive dependencies.

S. Interview Preparation

Question	Short answer	Evidence
Why FastAPI?	Typed, testable endpoints with dependency injection and Pydantic integration.	<code>main.py</code> , <code>routes.py</code> , <code>schemas.py</code>
Why PostgreSQL + pgvector?	One durable system stores chat, documents, metadata, and vector search.	<code>models.py</code> , <code>retriever.py</code>
How does RAG prevent hallucination?	Retrieve thresholded transcript evidence, prompt only that context, and return sources.	<code>rag_service.py</code>
Why all-MiniLM?	Small CPU-friendly embedding model with 384-D vectors suitable for local Docker.	<code>embedder.py</code> , migration
How does routing work?	Deterministic ordered term matching: artifact, Ship30, then RAG default.	<code>router.py</code>
How are skills separated?	Each skill owns its output prompt/shape while reusing RAGService.	<code>agent/skills</code>
Why provider abstraction?	Skills depend on one generate contract; Ollama/Claude transport can vary.	<code>provider.py</code> , factory
How does Ollama connect?	Backend sends JSON to <code>host.docker.internal:11434</code> through urllib.	<code>ollama_provider.py</code> , Compose
How is HTML protected?	Fence normalization, server-side defense-in-depth, and sandboxed iframe.	<code>artifact_skill.py</code> , <code>App.tsx</code>
How would you scale?	Cache the embedding model, use a worker/job for ingestion, add auth, indexes, observability, and provider timeouts/retries.	Current limits documented in README.

Final Master Checkpoint

Use these prompts for every technology while reading the code.

Technology	WHAT IS IT? / WHAT DOES IT DO?	WHY / HOW / WHERE?	RAG connection, failure, debug, interview
Python / virtualenv / pip	Interpreter and isolated dependency installer.	Host <code>myenv</code> ; Docker Python 3.11; pinned requirements.	Supplies all layers; debug versions/build; explain reproducibility.
FastAPI / Uvicorn	API framework / ASGI runtime.	Creates and serves app/routes.	Begins request flow; debug health/logs; explain typed API boundary.
Pydantic	Validation/serialization models.	Request/response schemas.	Protects prompt/API inputs; debug 422; explain contracts.
SQLAlchemy / Psycopg	Python DB toolkit / PostgreSQL driver.	Engine, sessions, ORM, SQL URL.	Persists corpus/chat; debug connection/transactions.
Alembic	Schema migration runner.	Creates tables/vector dimension history.	Prevents schema drift; run current/upgrade.
PostgreSQL / pgvector	Relational DB / vector extension.	Documents, chunks, embeddings.	Ranks evidence; debug dimension/index/query.
Sentence Transformers / PyTorch	Embedding model/runtime.	Normalized MiniLM vectors.	Enables semantic retrieval; debug model/download/shape.
LangChain splitter	Recursive text chunker.	1000 size, 200 overlap.	Defines retrieval units; debug chunk count.
python-dotenv	Loads local environment files.	main/database/Alembic.	Selects DB/provider; never print secret values.
Pytest / HTTPX	Test runner / TestClient transport dependency.	Unit, API, DB, controlled retrieval tests.	Protects contracts; debug focused test/image rebuild.
Claude Agent SDK	Optional async Claude execution SDK.	Executor query/options; external auth.	Alternative generation path; mock query in tests.
Ollama	Local chat model service.	JSON HTTP provider, default.	Final grounded generation; debug host networking/model.
LLMProvider	Abstract generation interface.	Ollama/Claude/HF implementations.	Decouples RAG from transport.
GrowthAgent/Router	Application orchestration and intent selection.	Deterministic routes to skills.	Connects request to RAG/generation.
RAG/Ship30/Artifact Skills	Explicit task boundaries.	Reuse RAG context, specialize prompts/output.	Answer, essay, or HTML artifact.
React/TypeScript	Frontend client.	Fetches API, shows sources/skill/artifact.	Displays final RAG result; debug API/CORS.
Docker	Reproducible services/runtime.	Backend/frontend/Postgres Compose.	Packages Python and networking; debug health/build.
Git/GitHub	Version control/remote collaboration.	Ignored secrets/data; no commit performed by this task.	Protects reproducibility; inspect status/diff/check-ignore.

Final study checkpoint: explain one complete request without opening the code: validated question → session transaction → deterministic skill → query embedding → pgvector threshold → source context → Ollama/Claude generation → assistant persistence → JSON/UI.

Generated from the current repository on 2026-09-14. This handbook is documentation only and does not change application code.